🧠 Learning How to Learn Programming

Fulllion Creative Works

Links, video titles and research references were checked in November 2026, and each video's summary was rechecked against what the video actually says.

Contents

1. The Method at a Glance
2. Plan: Decide What to Learn and Map It Out
3. Learn: Techniques That Make Things Stick
4. Build: Get Out of Tutorial Hell
5. Review: Notes, Reflection and Problem Solving
6. Motivation, Boredom and Following Through
7. Wellbeing Basics
8. Self-Studying Math
9. Books Worth Reading
10. Common Mistakes
11. Sources

---

1. The Method at a Glance

Learning to program isn't mainly about finding the perfect course. It's about a loop you repeat for every new topic:

Stage	What you do	Main tools
1. Plan	Pick one goal, get an overview, list the topics, and choose a small project	A short curriculum, a project plan (Section 2)
2. Learn	Study one topic at a time, then explain it and test yourself on it	The Feynman technique, active recall, spacing (Section 3)
3. Build	Use the topic in something you make, without following along step by step	A small project, then a bigger one (Section 4)
4. Review	Look back at what broke, what you looked up, and what to revisit	Short notes, a review list, quizzing yourself (Section 5)

Then go round again with the next topic. Each loop should feel a little hard. If it feels easy, raise the challenge; if it feels impossible, make the step smaller.

> **Learn in any order, but not all at once.** The original notes put it simply: you can pick up languages and tools in any order, but don't take on too many at the same time. One language, one project and one new tool at a time is plenty.

---

2. Plan: Decide What to Learn and Map It Out

2.1 Get an overview, then make a curriculum

In **"How To Learn Programming Languages and Frameworks FAST"** (Tech With Tim), Tim lays out five steps: get an overview, list the topics as your own curriculum, learn the topics one at a time, stop every few lessons to apply them, and repeat. He's clear that this makes you competent, not an expert. How he does each step:

- **Overview:** about half an hour of an expert's video, or 20 to 30 minutes of docs or a book's contents, to see what's in scope and what's too advanced for now.
- **Curriculum:** a numbered list of 20 to 30 topics, each with a few key points. For a new language that might be setup, tooling, output, input, arithmetic and data types. The first two steps together take about an hour.
- **Learn:** he mostly reads the official documentation with an editor open beside it, trying each example, and switches to a video (at double speed) when something won't click.
- **Apply:** he checks his understanding by explaining the topic (for him, filming a video). Exercises and small scripts work just as well.
- **Use what you already know.** When you learn a second language, look for how it **differs** from the first instead of relearning loops and data types from scratch.

To turn that into a plan:

1. **Get an overview.** Skim the official docs, a roadmap, or a short "what is X" video. You're learning the shape of the topic, not the details.
2. **List the topics** in the order you'll learn them. A roadmap from roadmap.sh is a good starting point.
3. **Pick a project that uses them.** A small, finished project beats a big, abandoned one.
4. **Set a realistic pace.** Decide how many hours a week you can really give it, and plan for that, not for your best-case week.

2.2 Plan your projects

"How I Plan My Coding Projects" (ForrestKnight) describes a middle way between "just wing it" and full professional planning, for personal web projects. His process:

1. **Sketch the layout** on paper: the pages and how a visitor moves between them. Leave colors, fonts and other details for later, because they're easy to change.
2. **Research before you code:** the stack, whether you need a database or an outside API, or, if you're rebuilding a game, its mechanics and scoring.

3. **Decide what to build first.** He usually starts where a visitor starts, such as the sign-up page.
4. **Define a minimum viable product (MVP):** the smallest version that does the core job, which you can show a few people and get feedback on before adding more.
5. **Then start coding,** figuring things out as you go.

He deliberately **skips** mockup tools, UML diagrams, user stories and sprints for solo projects. Teams use them to keep everyone on the same page, and you can learn them on the job. He also suggests **regular time slots instead of deadlines** (missed deadlines lead to stress and putting things off), and always **leaving yourself something to do next time**.

If you like a written list, keep it short: one sentence on what the project does, the MVP features, and a sketch of the screens on paper or in Excalidraw.

2.3 A sample study week

Adjust the hours to your life. The pattern matters more than the numbers.

Day	Session (about 1 to 2 hours)
Monday	Learn a new topic. Finish by explaining it in your own words (Section 3.1).
Tuesday	Build: use the topic in your project.
Wednesday	Quiz yourself on Monday's topic from memory, then keep building.
Thursday	Learn the next topic, mixing in a few practice problems on last week's.
Friday	Build.
Weekend	One review session: what broke, what you looked up, what to revisit. Then rest.

---

3. Learn: Techniques That Make Things Stick

A large 2013 review of ten common study techniques (Dunlosky and colleagues, in *Psychological Science in the Public Interest*) rated **practice testing** and **distributed practice** (spacing) as the most useful, and rated popular habits such as **rereading** and **highlighting** as low utility. The techniques below build on that work.

3.1 The Feynman technique

Named after the physicist Richard Feynman, this is "learn it by teaching it". **"Learn Code Faster with the Feynman Technique"** (ForrestKnight) breaks it into four steps: **pick** a topic, **learn and study** it, **teach** it to understand it, and **simplify**. He suggests doing it after every study or coding session.

1. **Pick** something you genuinely want to learn, then one concept within it, such as "what a closure is in JavaScript".
2. **Learn** it from a good source, such as the MDN docs.
3. **Teach it:** write it down as if for someone else (words, pictures or diagrams all count) and say it out loud like a teacher in front of a class. Write a tiny code example as part of the explanation. Wherever

you get stuck, go back to step 2, then try again.

4. **Simplify.** Rework the explanation until someone who has never coded could follow it. If it's wordy or confusing, start that part again.

It works because explaining forces you to pull ideas out of memory and connect them, which is exactly what rereading doesn't do.

3.2 Active recall (retrieval practice)

Active recall means getting information out of your head instead of putting it in again. Close the tutorial and write the code from memory, answer questions without looking, or explain a concept from scratch.

In a well-known experiment (Roediger and Karpicke, 2006), students who **tested themselves** on a passage remembered more of it two days and a week later than students who **reread** it, even though rereading looked better on a test five minutes later. That's the catch: retrieval feels harder and slower in the moment, which is why people avoid it.

Ways to practice it as a programmer:

- **Rebuild from memory.** After a tutorial, close it and rebuild the feature without looking. Look things up only when you're truly stuck, then try again.
- **Blank-page recall.** Write down everything you remember about a topic, then check what you missed.
- **Flashcards for facts, sparingly.** Syntax, shortcuts and terms suit flashcards; how to design a program doesn't.

3.3 Spaced repetition

Spacing means reviewing the same material over several sessions spread across days, instead of cramming it into one. A large review of the research (Cepeda and colleagues, 2006) found that spacing reliably helps memory, and that the best gap between reviews gets longer the longer you want to remember something.

In practice:

- Review a new topic the next day, then a few days later, then a week or two later.
- Flashcard apps such as Anki schedule this for you.
- Coming back to a concept in a new project is also spacing, and it's more useful than a flashcard.

3.4 The catch: recall isn't the whole story

"The PROBLEM with Active Recall and Spaced Repetition (Truth Behind Studying Smarter)" (Justin Sung) is a 40-minute video from a doctor turned learning coach. His argument:

- **Studying isn't the same as learning.** Studying is the activity (reading, notes, flashcards). Learning is what happens in your head, and some activities produce far more of it than others.
- **Memory has two halves: encoding** (getting information into long-term memory in the first place) and **retrieval** (getting it back out). Active recall and spaced repetition mostly train retrieval. You can't retrieve what was never encoded well.

- **Good encoding feels effortful.** He calls this cognitive load: the slightly confused feeling of asking how a new idea connects to what you know, and how it's similar or different. If studying feels boring or sleepy, you're probably not encoding much. Too much load, though, overwhelms you.
- **Recall and spacing have diminishing returns.** They slow down forgetting, which helps a lot if you're doing no review at all, but you have to repeat everything several times to keep it, which doesn't scale to large amounts of material.
- **Why the techniques are so hyped:** they're easy to explain, and people who failed with them rarely make videos about it.

The point worth taking from it: recall and repetition help you **keep** what you've learned, but they can't make up for **understanding it badly in the first place**. Drilling flashcards of facts you never connected together can leave you able to recite things you can't use.

A note of caution. His claims that most people using flashcard systems don't do well, and that more recall can make strong students worse, come from his coaching experience rather than studies he cites in the video. The large reviews in this section still rate practice testing and spacing as among the most useful techniques. The video also promotes his paid course. Treat it as "recall plus understanding", not "recall is useless".

Jump to: studying versus learning · how memory works: encoding and retrieval · cognitive load · the forgetting curve and diminishing returns · why the techniques are hyped

What that means for you:

1. **Understand first, then drill.** When you meet a new idea, ask how it connects to what you know, why it works, and when you'd use it.
2. **Recall relationships, not just facts.** "Why would I use a `Map` instead of an object here?" is a better question than "what's the syntax for `Map`?".
3. **Use building as your main review.** Applying an idea in a project tests understanding and memory at the same time.

3.5 Interleaving

Interleaving means mixing different kinds of problems in one session instead of doing all of one kind and then all of the next. A 2019 meta-analysis (Brunmair and Richter) found a moderate benefit overall, but it depended heavily on the material: it helped most when the things being learned were easy to confuse with each other, and it didn't help with every kind of material.

For programmers, it helps when you must learn to **choose** between approaches. Instead of 10 loop exercises followed by 10 recursion exercises, mix them, so that each time you first have to decide which tool fits.

Tina Huang adds three practical cautions in **"Tips to Learn Technical Things Faster"** (Tina Huang), which fit the research above:

- **Don't interleave at the very start.** Get a baseline in one topic first.

- **Don't switch every few minutes,** or it becomes multitasking. She leaves at least an hour or two between topics, and often does one topic a day.
- **Mix related topics** (such as two ways of solving the same kind of problem) for the biggest benefit.

3.6 Deliberate practice

Deliberate practice is the psychologist Anders Ericsson's term for practice designed to improve a specific weakness, at the edge of what you can do, with fast feedback. Andy Sterkowitz recommends Ericsson and Pool's book *Peak* in **"How to Learn to Code FAST (Do This or Keep Struggling)"** (Andy Sterkowitz), along with Scott H. Young's *Ultralearning*. He opens by saying that hours alone don't make you better: he has played thousands of hours of casual beach volleyball and barely improved. His version of deliberate practice has three parts:

1. **A clear goal for each session.** Not "do some coding", but "today I'll just practice understanding what LeetCode questions are asking, without writing code".
2. **Full focus,** with no video or podcast in the background.
3. **Working outside your comfort zone,** such as actually using closures in a problem, not just reading another article about them.

His other two principles:

- **Attack your weaknesses directly.** He got better at a video game by drilling the one move he was bad at instead of playing more matches. For code: keep a running list of weak spots (asynchronous JavaScript and promises are a common one), and when you have time, drill them, for example by searching for interview questions on that topic and working through them.
- **Manage your energy like an athlete.** Sleep 7 to 8 hours, take breaks when you hit a wall (he describes this as letting "diffuse thinking" work in the background), split a long study day into blocks of about two hours, and get regular moderate exercise. Section 7 has the health guidance behind this.

Two honest caveats:

- A 2014 meta-analysis (Macnamara, Hambrick and Oswald) found deliberate practice mattered a lot in games, music and sports, but explained much less of the difference between people in education and professions. It's important, not magic.
- Hours alone don't count. An hour spent on the thing you find hardest, with feedback from tests, a reviewer or a running program, is worth more than three comfortable ones.

3.7 "Desirable difficulties"

Researchers at UCLA's Bjork Learning and Forgetting Lab use the term **desirable difficulties** for study conditions that make practice feel harder but improve long-term learning: spacing instead of cramming, testing yourself instead of rereading, and varying how you practice. If a study session feels smooth and easy, it may not be doing much.

3.8 More videos on learning faster

"Tips to Learn Technical Things Faster" (Tina Huang) gives five study habits from a data scientist:

1. **Active recall,** her way: after reading a section, summarize it out loud in your own words. She finds talking faster than writing questions.
2. **Environment design,** an idea she credits to James Clear's *Atomic Habits*: make the good habit easy and the distraction hard, instead of relying on willpower. Her examples are locking her phone in a timed box and studying at a library.
3. **Switching between focused and diffuse thinking,** from Barbara Oakley's *Learning How to Learn* course: when you're stuck, walk away for a while and let your mind work on it in the background.
4. **Interleaving,** with the cautions in Section 3.5.
5. **Priming:** skim new material before you study it, to see its structure and key terms. On a technical course she sometimes starts the final project first, then goes back to learn each part as she hits it.

"How To Self Study Technical Things" (Nick White) puts **understanding ahead of volume**, then gives four steps:

1. **Find an angle that interests you,** such as what the topic is used for in the real world.
2. **Learn the building blocks first.** He tried to jump from basic Python syntax to computer vision and couldn't follow it, because it assumed classes, libraries and the command line. If you can't understand something, look for the prerequisite you're missing.
3. **Practice daily,** not once a week, so you aren't relearning last week's material.
4. **Teach it or discuss it** with someone, which shows whether you really understand it.

---

4. Build: Get Out of Tutorial Hell

Tutorial hell is the loop of watching tutorial after tutorial, feeling like you understand, and then freezing in front of an empty editor. Following along feels productive, but the tutorial is doing the hard parts: deciding what to build, breaking it down, and recovering from mistakes.

Kevin Powell's one-minute short **"Don't learn web dev this way"** (Kevin Powell) describes exactly this: video after video and long online courses, then no idea where to start. He compares it to learning to skateboard by watching people at the skate park. Videos, blogs and books are for getting unstuck; the learning happens when you build.

How to get out:

1. **One tutorial, then one project of your own.** After each tutorial, build something similar but different without it: a different topic, an extra feature, a different layout.
2. **Change the tutorial project.** Add a feature the tutorial didn't cover. The moment you leave the script, you start really learning.
3. **Build from a brief, not a video.** Sites such as Frontend Mentor, roadmap.sh projects and The Odin Project give you a goal and leave the "how" to you. More are in Coding Practice and Interview Prep.
4. **Look up small things, not whole solutions.** Search for "how to sort an array in JavaScript", not "to-do app tutorial".

5. **Finish things.** A small finished project teaches you deploying, debugging and polishing, which half-finished ones never do.

Projects don't have to be websites. Small games are a great way out of tutorial hell: a game jam gives you a theme, a deadline and a finished thing to show, and Learn Game Development has a scope checklist for a first game. Learn 3D Modeling uses the same method outside programming: a guided first hour, then a ladder of projects that each add one skill.

"Learn to Code for FREE: Quick, Easy & Fun Way" (PIRATE KING), from a software engineer who has worked at Microsoft, Amazon and eBay, argues that **building a website** is the best way to learn: the way to get better at coding is to code. His advice:

- **Learn JavaScript first,** with HTML and CSS, and Python if you want a second language. Free YouTube tutorials are enough to start.
- **Don't build a project that only creates, reads, updates and deletes records** for the sake of it. Either solve a real problem (a simple site for a small local business, or something using a free public API) or build something useful to you. His example is a bookkeeping app for his household that pulls in bank transactions automatically.

> **What's changed since the video:** it suggests hosting the back end free on Heroku, but Heroku ended its free plans on November 28, 2022. For current hosting options and today's front-end picks, see Learn Front-End Web Development and Full Stack Web Development and Stacks.

"Why 95% of Self-Taught Programmers Fail (Honest Advice)" (Andy Sterkowitz) is a useful self-check if you feel stuck. It names four early warning signs:

Warning sign	His fix
No simple learning process. Without a school curriculum, people pick random things and stay busy without progressing	Set a fixed study schedule, and decide **before** each session what you'll work on, so you don't drift to the easiest task
Theory over application. He learned to make a cappuccino at Starbucks by making three or four, not by watching hours of other people	Learn the basics, then build small projects. Employers hire for what you can **do** with what you know
Fear of mistakes. If changing an old project scares you, that's a sign	Treat bugs as feedback on what you don't yet understand, read code line by line, use the debugger, and use Git commits and branches so nothing is ever lost
No date to start applying. You'll never feel ready, and waiting is self-sabotage	Pick a "drop-dead date" to start applying for jobs, whatever you feel like on the day

> **What's changed since the video:** it says companies were in dire need of developers. That reflects the market when it was made. Since then, hiring for software developers has been much tighter: Indeed's Hiring Lab reported US software development job postings about 32% below their February 2020 level in January 2026. Setting a date is still good advice, but expect a longer search. See Web Developer Careers, Portfolio and Freelancing.

> **About AI coding assistants:** they can explain errors and suggest code, but if they write everything, you skip the practice that makes you able to do it yourself. Try first, then ask, and make sure you understand anything you keep. See Open Source and Developer Communities for using them responsibly.

---

5. Review: Notes, Reflection and Problem Solving

5.1 Notes that help

The original notes included two opposite views, and both have a point:

- **"How I take notes - Tips for efficient note taking that speeds up learning"** (Tina Huang) builds study techniques into the notes themselves:
  - **Before:** review your previous notes, then **prime** by skimming the next section for its main topics.
  - **During:** write only **high-level concepts**, your **own insights and connections**, and **full worked examples**. For details, note where to find them (a page number, or a video title and timestamp) instead of copying them out. She calls this a "framework of references". If you tend to write everything down, play lectures at 1.5 to 3 times speed so you can only catch what matters.
  - **After:** say what each section covered out loud without looking, then write a few summary points. That's active recall, and the summary doubles as your quick review later.
- **"Taking Notes is a WASTE OF TIME When You're Learning To Code! DO THIS INSTEAD!"** (Dorian Develops) says notes he took while learning mostly went unread, and that coding is about applying knowledge, not memorizing for a test. Instead:
  - **For your first month or two,** don't bother with notes. Just work through your course.
  - **Once you build projects, over-comment your code,** even code you copied, explaining each part. Writing the explanation forces you to understand it, and you'll learn later which comments are worth keeping.
  - **Start every project with a README** in Markdown, saying what it does and what it uses. Employers will see your GitHub, not your notebook.
  - **Only jot something down** when the same thing has tripped you up about three times.

A middle path that fits the research in Section 3:

- **Don't copy the tutorial.** Copying is rereading with extra steps.
- **Write short notes in your own words** after you've tried something, not while you watch.
- **Keep working code as your notes.** A folder or repository of small, commented examples ("how I did fetch with error handling") is more useful than pages of text.
- **Keep a "things I looked up" list.** Anything you search for twice goes on it, and that list becomes your review material.
- **Turn notes into questions.** "What does `await` do?" is something you can quiz yourself on later.

5.2 Reflect after each session

At the end of a session, spend five minutes answering:

1. What did I build or learn?
2. What broke, and how did I fix it?
3. What did I have to look up?
4. What's the next small step?

The last answer is the first thing you do next time, so you never start cold.

5.3 Problem solving

"How To Be a Better Problem Solver" (Namanh Kapur) splits problem solving into three parts:

- **Matching:** most problems have already been solved, so get good at recognizing which known solution fits. He suggests learning, in order: common **algorithms** (he recommends Jeff Erickson's *Algorithms*, which is free to download under a Creative Commons license), the **data structures** they use, **design patterns** (company engineering blogs are a good source), and the real tools that implement those ideas in industry, such as Redis for caching or a message queue. Also keep your own record of problems you've solved and how. See Computer Science Fundamentals.
- **Debugging:** search the error message (often it's your setup or a library, not your logic), write tests, try several suggested fixes (search a library's GitHub issues too), use print statements or a debugger to check the code takes the path you expect with the values you expect, then double-check the usual suspects: `and` versus `or`, loops, `if` / `else` order, what you're mutating, and what you're passing in. If all else fails, rewrite it from a clean start.
- **Innovating:** solving problems nobody has solved yet. In his view, that's mostly academic research, and few working developers need to worry about it.

A habit that helps every time: **say the problem out loud in one sentence before you write code**, then write the steps as comments, then fill in the code under each comment.

---

6. Motivation, Boredom and Following Through

6.1 Following through

"The Real Reason You Never Follow Through (And How to Fix It)" (HealthyGamerGG) treats follow-through as a skill anyone can build, not a trait you either have or don't. Its key idea: the hard part usually isn't staying on the task, it's the **craving to switch** to something else. Your mind gets used to whatever you're doing (the way you stop noticing your shirt), then wants something new. Instead of fighting that with willpower, it teaches **urge surfing**, a technique the presenter, a psychiatrist, uses in addiction treatment: notice the urge to switch (and the excuses that come with it, such as "this is boring, do it tomorrow"), don't act on it, and let it fade, because urges don't last. Start with the easier urges, say a quarter of them, rather than your strongest one.

Practical versions:

- **Make starting tiny.** "Open the project and fix one bug" is easier to begin than "study for two hours".
- **Remove the easiest distraction** before you start: phone in another room, game launcher closed.
- **Track streaks loosely.** Missing a day is normal. Missing two in a row is the signal to make the next session smaller, not to quit.

6.2 Boredom, repetition and finding the right difficulty

A short Facebook reel by **Josh Terry**, captioned "It's a skill" (it needs a Facebook login to view), is about sitting with boredom. His argument: mastering a skill takes lots of repetition **below** your maximum level, and that's boring. Our instinct is to chase progress by learning something new, because that's how a skill feels at the start. To master what you already have, redirect that drive: give practice at a comfortable level your full attention, so you're maxing out your **focus** even when you're not maxing out the **difficulty**.

That fits the research, with one nuance:

- **Boredom while learning is normal.** Repetition, setup and reading docs are part of the job. Feeling bored doesn't mean you've chosen the wrong path.
- **Practice mostly just below your limit.** A 2019 paper ("The Eighty Five Percent Rule for optimal learning", Wilson and colleagues, *Nature Communications*) found that some learning systems learn fastest when they get about 85% right, so practice should be mostly doable with some misses. It modeled machine learning algorithms and simple yes-or-no tasks, not people learning to code, so treat it as a rule of thumb, not a law.
- **Full attention is what makes repetition count.** Focus is one of the parts of deliberate practice in Section 3.6. Rewriting a familiar pattern while thinking about why each line is there is practice; doing it on autopilot isn't.
- **Adjust the difficulty, not your goal.** If you're bored and coasting, add a constraint or a feature. If you're failing most of the time, cut the task in half.
- **Do the most you can with what you know now.** You can build real, useful things with basic HTML, CSS and JavaScript. You don't need to wait until you "know enough".

6.3 Leaving your comfort zone

"Comfort Will Ruin Your Life" (Better Than Yesterday) argues for doing something difficult every day. It describes three zones: the **comfort zone** (which slowly shrinks if you never leave it), the **growth zone** (uncomfortable, and where progress happens), and the **danger zone** (too much at once, which leads to injury, burnout or quitting). To move into the growth zone:

1. **Raise one thing at a time:** how **often** you do it, how **intensely**, or for how **long**. For learning a skill, often beats long: ten minutes daily usually does more than one long weekly session.
2. **Go up in small steps.** Most people fail by jumping straight to the danger zone, then decide they can't change. Small wins build a feedback loop that keeps you going.
3. **Pull back slightly when it gets overwhelming,** to just above where you were, and let your comfort zone catch up before pushing again.
4. **Don't push in every area at once.** Rest in some while you grow in others.

For a programmer, the daily difficult thing might be reading an error message properly instead of pasting it somewhere, writing a test, or asking a question in a community.

6.4 When trying harder makes it worse

"The Harder You Try, The Worse It Gets | Law of Reversed Effort" (Einzelgänger) explains the "law of reversed effort", an idea from Aldous Huxley also called the backwards law: some things get harder the more you force them, like trying to fall asleep. It draws on Viktor Frankl's observation that anxious over-focus on an outcome can cause the very thing you fear (his technique, "paradoxical intention", is to wish for it instead), and on the idea of flow, where skilled action seems to happen by itself. Its conclusion is a middle path: you need conscious effort to **build** a skill, and a calmer, less forced state to **use** it well.

The video doesn't mention programming, but the idea applies. Staring at the same bug for three hours is a common example. Take a walk, sleep on it, or explain it to someone (Section 3.1), and the answer often turns up when you stop forcing it. Andy Sterkowitz and Tina Huang give the same advice (Sections 3.6 and 3.8). This isn't a reason to stop working; it's a reason to take breaks.

---

7. Wellbeing Basics

> **This isn't medical advice.** It's general guidance from public health bodies. If something here doesn't fit your situation, or you're worried about your health, talk to a doctor or another qualified professional.

7.1 Sleep

A consensus statement from the American Academy of Sleep Medicine and the Sleep Research Society says adults should sleep **7 or more hours a night on a regular basis**. Late-night cramming trades away the sleep that helps you think clearly the next day. A regular bedtime is one of the cheapest study aids there is.

7.2 Breaks

- **Work in focused blocks.** The Pomodoro Technique, created by Francesco Cirillo in the 1980s, uses short timed blocks (traditionally 25 minutes) followed by short breaks. Any similar rhythm works.
- **Step away from the screen** during breaks: stand up, look at something far away, get water.
- **Stop when you're stuck in a loop.** Section 6.4 applies here too.

7.3 Exercise and movement

The World Health Organization recommends that adults get **at least 150 minutes of moderate-intensity physical activity a week** and limit time spent sitting. For people who sit at a desk all day, even short walks between study blocks help.

7.4 Burnout signs

The World Health Organization describes burn-out, in its ICD-11 classification, as an occupational phenomenon (not a medical condition) caused by chronic workplace stress that hasn't been managed, with three signs:

- **exhaustion**, or feeling drained of energy;
- **mental distance** from the work, or feeling negative or cynical about it;
- **feeling less effective** at it.

Strictly, the WHO term applies to work, but people learning intensively often describe the same pattern. If you notice it, **reduce your load before it reduces you**: shorter sessions, a day off each week, a smaller project, and a realistic timeline. If those signs persist or you feel low most days, talk to a professional.

---

8. Self-Studying Math

Most web development needs very little math beyond arithmetic and logic. Computer Science Fundamentals explains which math matters for which kinds of programming.

If you want to go deeper, **"How to self study pure math - a step-by-step guide"** (Aleph 0) walks through a typical three-year undergraduate pure math sequence, with one main book (and often a free lecture series) per subject. It favors books with lots of examples and exercises, because those suit self-study. Its picks include:

Subject	Book or resource from the video
Linear algebra	*Linear Algebra Done Right* by Sheldon Axler, plus his short video playlist. The fourth edition is free to download under a Creative Commons license
Real analysis (rigorous calculus)	*Understanding Analysis* by Stephen Abbott, plus Francis Su's recorded lectures
Point-set topology	The University of Toronto's online notes and their long problem list
Complex analysis	*Visual Complex Functions* (phase portraits) for intuition, then Serge Lang's *Complex Analysis*
Group theory	Benedict Gross's recorded lectures, and I. N. Herstein's *Topics in Algebra*
Galois theory	Tom Leinster's free lecture notes
Algebraic topology	Allen Hatcher's *Algebraic Topology*, which is free to download from his site, plus Pierre Albin's lectures that follow it

The same method from this guide applies: do the exercises (active recall), come back to topics over time (spacing), and explain proofs in your own words (the Feynman technique).

---

9. Books Worth Reading

9.1 From "6 MUST READ Software Engineering Books 2022"

In **"6 MUST READ Software Engineering Books 2022"** (Keep On Coding), the presenter flips through six books that the publisher, Pearson Addison-Wesley, sent him, and gives a quick verdict on each. The book links in its description are affiliate links. His notes:

Book	Author	What he says about it
Modern Software Engineering	David Farley	Best practices: working iteratively, managing complexity, loose coupling and separation of concerns. Few code examples
The Pragmatic Programmer	David Thomas and Andrew Hunt	A classic of short, practical topics on habits, tools and debugging, and the source of "don't repeat yourself" (DRY). One every programmer would benefit from
Principles of Web API Design	James Higginbotham	The whole API design process, from planning to refining, with lots of diagrams but no implementation code. Suits planners as much as coders
Clean Code	Robert C. Martin	Shows bad code, explains why, then fixes it. Heavy on Java and object-oriented programming (see the notes below)
Python Distilled	David M. Beazley	Not for your first language. It takes an intermediate Python developer toward advanced
Code That Fits in Your Head	Mark Seemann	Grows one code base from nothing to deployment, then covers maintaining it. Examples in C#

9.2 A short classic reading list

- **The Pragmatic Programmer, 20th Anniversary Edition** by David Thomas and Andrew Hunt (2019). Short, practical chapters you can read in any order. A good first pick.
- **Code: The Hidden Language of Computer Hardware and Software** by Charles Petzold. A readable explanation of how computers work, from simple codes and circuits up to a working processor. The expanded second edition came out in 2022, with an interactive companion site, CodeHiddenLanguage.com.
- **A Philosophy of Software Design** by John Ousterhout (second edition, 2021). A short book on managing complexity, built around designing "deep" modules with simple interfaces.
- **Clean Code** by Robert C. Martin. Very influential, and a second edition came out in 2025. **Read it critically.** Some of its advice, such as keeping functions extremely short, is widely disputed. Good counterpoints are qntm's essay "It's probably time to stop recommending Clean Code", Casey Muratori's "'Clean' Code, Horrible Performance", and a written discussion between John Ousterhout and Robert Martin on where their books disagree.
- **Grokking Algorithms, Second Edition** by Aditya Bhargava (2024). An illustrated, friendly introduction to algorithms, with Python examples. Pairs well with Computer Science Fundamentals.

> **How to read a programming book:** don't just read it. After each chapter, try one idea in your own code, and write down one thing you'd do differently. That's Sections 3 and 4 applied to books.

---

10. Common Mistakes

Mistake	Why it hurts	What to do instead
Watching tutorials back to back	You recognize code but can't write it	Follow every tutorial with a project you build alone (Section 4)
Rereading and highlighting	Feels productive, but helps memory little	Test yourself and explain topics from memory (Section 3)
Learning five things at once	Nothing gets deep enough to use	One language, one project, one new tool at a time
Cramming the night before	Spacing beats cramming for long-term memory	Short, repeated sessions across days
Copying full notes from videos	Slow, and it's passive	Short notes in your own words, plus working code examples
Waiting until you "know enough" to build	You never feel ready	Build something small with what you know now
Studying through exhaustion	Sleep loss and burnout slow you down	Regular sleep, breaks and days off (Section 7)

---

11. Sources

Videos from the original notes

- Tech With Tim, "How To Learn Programming Languages and Frameworks FAST": https://www.youtube.com/watch?v=RPrBGrQsIek
- ForrestKnight, "How I Plan My Coding Projects": https://www.youtube.com/watch?v=qS4mvrWWO_M
- ForrestKnight, "Learn Code Faster with the Feynman Technique": https://youtu.be/vWNa15pjzi4
- Justin Sung, "The PROBLEM with Active Recall and Spaced Repetition (Truth Behind Studying Smarter)": https://www.youtube.com/watch?v=--Hu2w0s72Y
- Andy Sterkowitz, "How to Learn to Code FAST (Do This or Keep Struggling)": https://youtu.be/mw0LGzIKvoo
- Andy Sterkowitz, "Why 95% of Self-Taught Programmers Fail (Honest Advice)": https://www.youtube.com/watch?v=ueXjGMrmn8k
- Tina Huang, "Tips to Learn Technical Things Faster": https://www.youtube.com/watch?v=9rGweuLdook
- Tina Huang, "How I take notes - Tips for efficient note taking that speeds up learning": https://youtu.be/q6_9mdCkQhE
- Dorian Develops, "Taking Notes is a WASTE OF TIME When You're Learning To Code! DO THIS INSTEAD!": https://youtu.be/VCWzQpUwsaw
- Nick White, "How To Self Study Technical Things": https://www.youtube.com/watch?v=UkBq1waNmtQ
- Namanh Kapur, "How To Be a Better Problem Solver": https://youtu.be/CadQhoBUvlY
- PIRATE KING, "Learn to Code for FREE: Quick, Easy & Fun Way": https://youtu.be/S-vKRPLdEGw

- Kevin Powell, "Don't learn web dev this way": https://youtube.com/shorts/cMtYwPcwcBU
- HealthyGamerGG, "The Real Reason You Never Follow Through (And How to Fix It)": https://youtu.be/0hePvDEOZYs
- Better Than Yesterday, "Comfort Will Ruin Your Life": https://youtu.be/F5-qkgfBeuU
- Einzelgänger, "The Harder You Try, The Worse It Gets | Law of Reversed Effort": https://youtu.be/-IlG32Pb43g
- Aleph 0, "How to self study pure math - a step-by-step guide": https://youtu.be/byNaO_zn2fI
- Keep On Coding, "6 MUST READ Software Engineering Books 2022": https://youtu.be/9t7R4GBUAq0
- Josh Terry, "It's a skill" (Facebook reel, login needed): https://www.facebook.com/reel/617775599977814

Research and guidance

- Dunlosky, Rawson, Marsh, Nathan and Willingham (2013), "Improving Students' Learning With Effective Learning Techniques", *Psychological Science in the Public Interest*: https://www.psychologicalscience.org/publications/journals/pspi/learning-techniques.html
- Roediger and Karpicke (2006), "Test-Enhanced Learning", *Psychological Science*: https://psychology.ecu.edu/wp-content/pv-uploads/sites/216/2019/03/Roediger-Karpicke-2006.pdf
- Cepeda, Pashler, Vul, Wixted and Rohrer (2006), "Distributed Practice in Verbal Recall Tasks", *Psychological Bulletin*: https://www.evullab.org/pdf/CepedaPashlerVulWixtedRohrer-PB-2006.pdf
- Brunmair and Richter (2019), "Similarity matters: A meta-analysis of interleaved learning and its moderators", *Psychological Bulletin*: https://www.psychologie.uni-wuerzburg.de/fileadmin/06020400/2019/Brunmair_Richter_in_press__2019_META-ANALYSIS_OF_INTERLEAVED_LEARNING.pdf
- Macnamara, Hambrick and Oswald (2014), "Deliberate Practice and Performance in Music, Games, Sports, Education, and Professions: A Meta-Analysis", *Psychological Science*, discussed in "Facing facts about deliberate practice": https://pmc.ncbi.nlm.nih.gov/articles/PMC4101876/
- Bjork Learning and Forgetting Lab (UCLA), research and desirable difficulties: https://bjorklab.psych.ucla.edu/research/
- American Academy of Sleep Medicine and Sleep Research Society, adult sleep duration consensus: https://aasm.org/resources/pdf/pressroom/adult-sleep-duration-consensus.pdf
- World Health Organization, physical activity fact sheet: https://www.who.int/news-room/fact-sheets/detail/physical-activity
- World Health Organization, "Burn-out an 'occupational phenomenon'": https://www.who.int/news/item/28-05-2019-burn-out-an-occupational-phenomenon-international-classification-of-diseases
- Francesco Cirillo, creator of the Pomodoro Technique: https://www.francescocirillo.com/ and https://www.pomodorotechnique.com/

Books and reviews

- The Pragmatic Programmer, 20th Anniversary Edition (Pragmatic Bookshelf): https://pragprog.com/titles/tpp20/the-pragmatic-programmer-20th-anniversary-edition/
- Code: The Hidden Language of Computer Hardware and Software, second edition (Pearson): https://www.pearson.com/store/en-us/p/code-the-hidden-language-of-computer-hardware-and-software/P200000007298/9780137909100
- A Philosophy of Software Design (John Ousterhout): https://web.stanford.edu/~ouster/cgi-bin/book.php
- Clean Code, second edition (Pearson): https://www.pearson.com/en-us/subject-catalog/p/clean-code-a-handbook-of-agile-software-craftsmanship-2nd-edition/P200000013239
- qntm, "It's probably time to stop recommending Clean Code": https://qntm.org/clean
- Casey Muratori, "'Clean' Code, Horrible Performance": https://www.computerenhance.com/p/clean-code-horrible-performance
- John Ousterhout and Robert Martin, A Philosophy of Software Design vs Clean Code: https://github.com/johnousterhout/aposd-vs-clean-code
- Grokking Algorithms, Second Edition (Manning): https://www.manning.com/books/grokking-algorithms-second-edition

Practice and project sites

- roadmap.sh: https://roadmap.sh/ and projects: https://roadmap.sh/projects
- Frontend Mentor: https://www.frontendmentor.io/
- The Odin Project: https://www.theodinproject.com/
- Anki: https://ankiweb.net/
- Excalidraw: https://excalidraw.com/

Other sources checked for this guide

- Wilson, Shenhav, Straccia and Cohen (2019), "The Eighty Five Percent Rule for optimal learning", *Nature Communications*: https://pmc.ncbi.nlm.nih.gov/articles/PMC6831579/
- Jeff Erickson, *Algorithms* (free, CC BY 4.0): https://jeffe.cs.illinois.edu/teaching/algorithms/
- Barbara Oakley and Terrence Sejnowski, *Learning How to Learn* (Coursera): https://www.coursera.org/learn/learning-how-to-learn
- Heroku, removal of free product plans FAQ: https://help.heroku.com/RSBRUH58/removal-of-heroku-free-product-plans-faq
- Indeed Hiring Lab, US tech hiring update for Q4 2025 (software development postings): https://d341ezm4iqaae0.cloudfront.net/hiringlaborg/2026/02/20105029/HiringLab-Tech_Q4_1.pdf
- Sheldon Axler, *Linear Algebra Done Right*, fourth edition (open access): https://linear.axler.net/
- Allen Hatcher, *Algebraic Topology* (free online edition): https://pi.math.cornell.edu/~hatcher/AT/ATpage.html

Latest version: github.com/ImmuneMoon/Learning-How-to-Learn-Programming

If this helped, you can support Fulllion Creative Works through PayPal or Ko-fi.

© 2026 Fulllion Creative Works. All rights reserved.